

# AIC313/CS378: Introduction to Generative Models

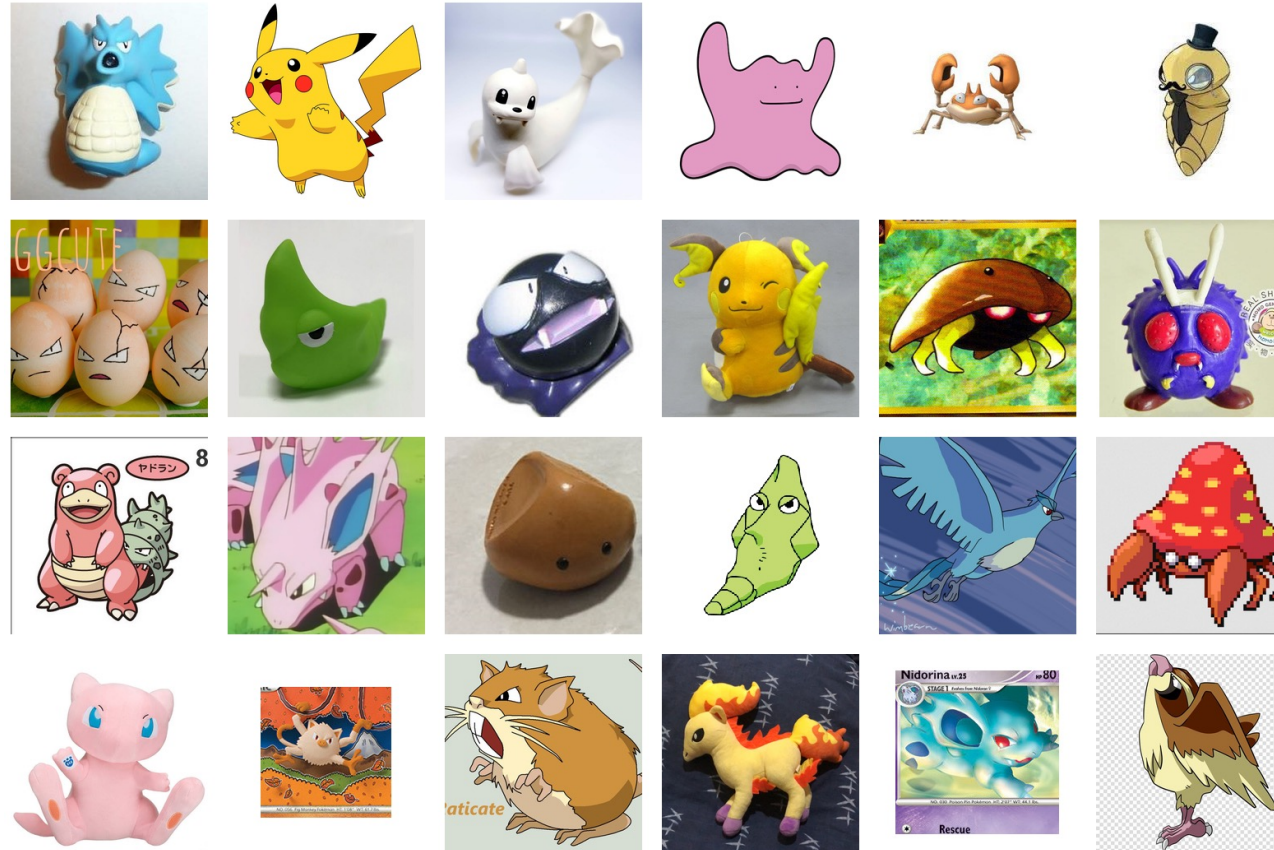
## FastGen Competition

MINGUE PARK

Fall 2026  
KAIST

# Generate Pokemon!

TL;DR: Your goal is to generate the Pokemon images FAST!



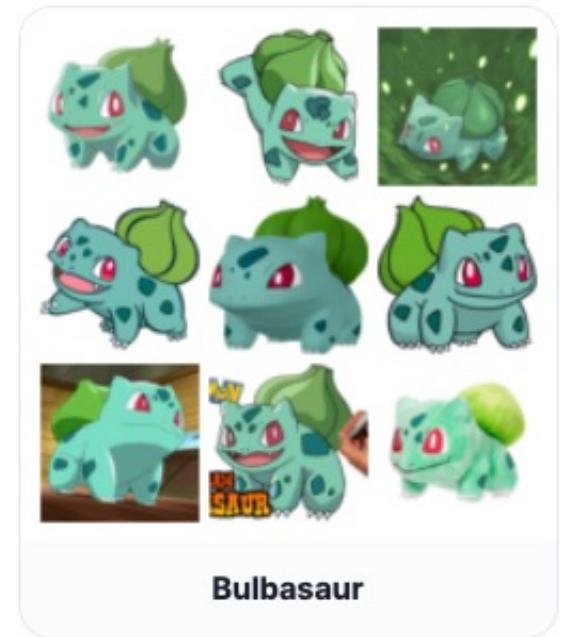
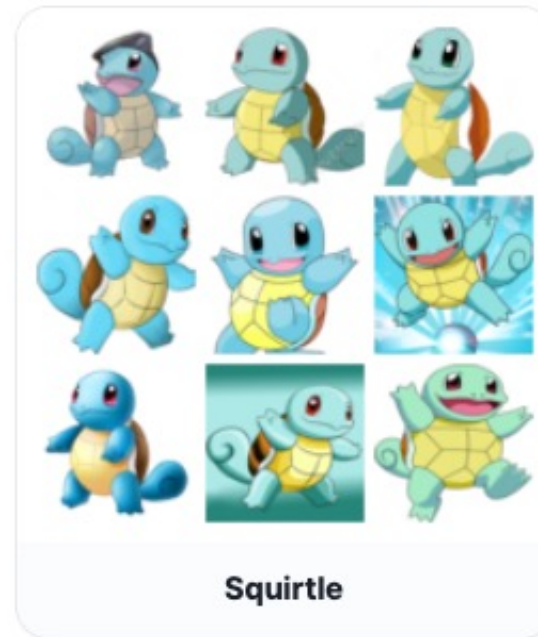
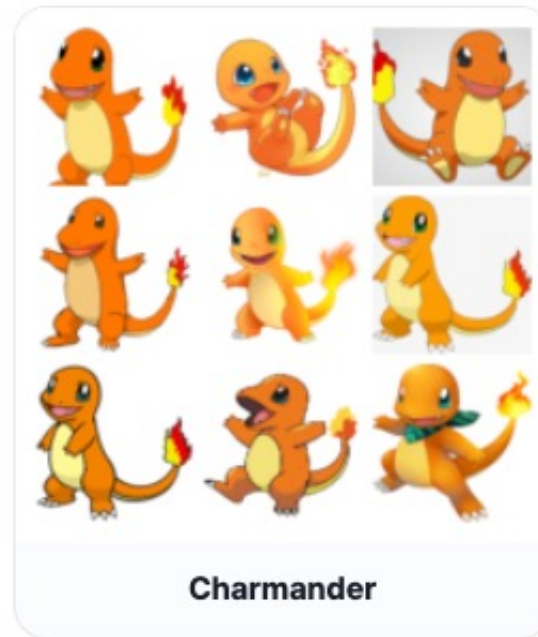
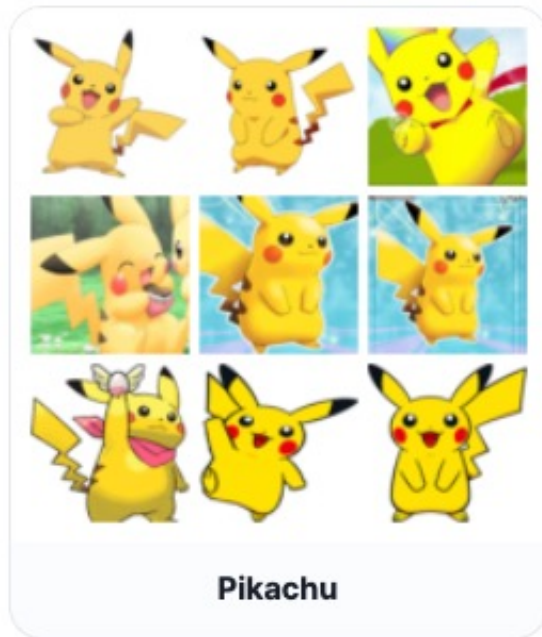
<https://github.com/KAIST-Visual-AI-Group/AIC313-Project-FastGen>

# Goal

- You will implement and train your own **image generation models**.
- The goal is to achieve high-quality image generation **with only a few sampling steps**.
- You are encouraged to explore advanced techniques for few-step generation.

# Pokemon Generation One Dataset

It consists of 20,099 images across 151 Pokemon categories.



<https://www.kaggle.com/datasets/bhawks/pokemon-generation-one-22k>

# Pokemon Generation One Dataset

- We will provide the training / validation split for the dataset.
  - Training set: 17,079 images, validation set: 151 categories  $\times$  20 = 3,020 images
- You are only allowed to use the training split for training your model. The validation split will be used for the evaluation.
- We use 64 $\times$ 64 down-sampled images for computation efficiency.

# Generation with a Few Steps

We will generate images using various generative models with only a few sampling steps.

# Evaluation

- Your model will be evaluated at NFE=1, NFE  $\leq$  4.

NFE: Number of Function Evaluations (=Sampling Steps)

- To assess the quality of generated images, we use Fréchet Inception Distance (FID) as an evaluation metric. The validation split will be used for the reference set.

# Fréchet Inception Distance (FID)

Heusel et al., GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium, NeurIPS 2017.

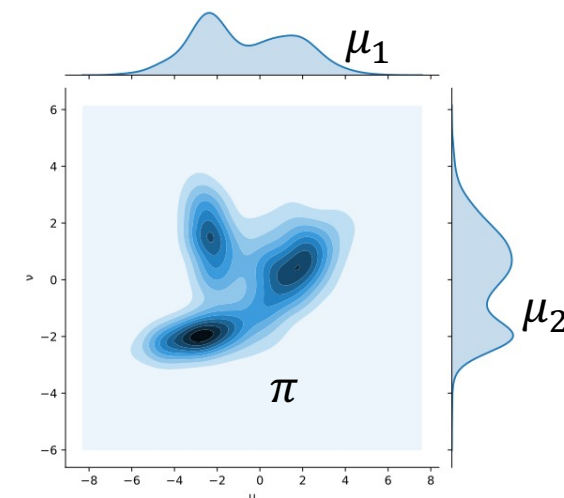
# $p$ -Wasserstein Distance

It measures the distance between two probability distributions.

a.k.a (in a discrete setup) **Fréchet** distance / Earth movers' distance

$$W_p(\mu, \nu) := \inf_{\pi \in \Gamma(\mu, \nu)} \left( \mathbb{E}_{(x,y) \sim \pi} d(x,y)^p \right)^{1/p}$$

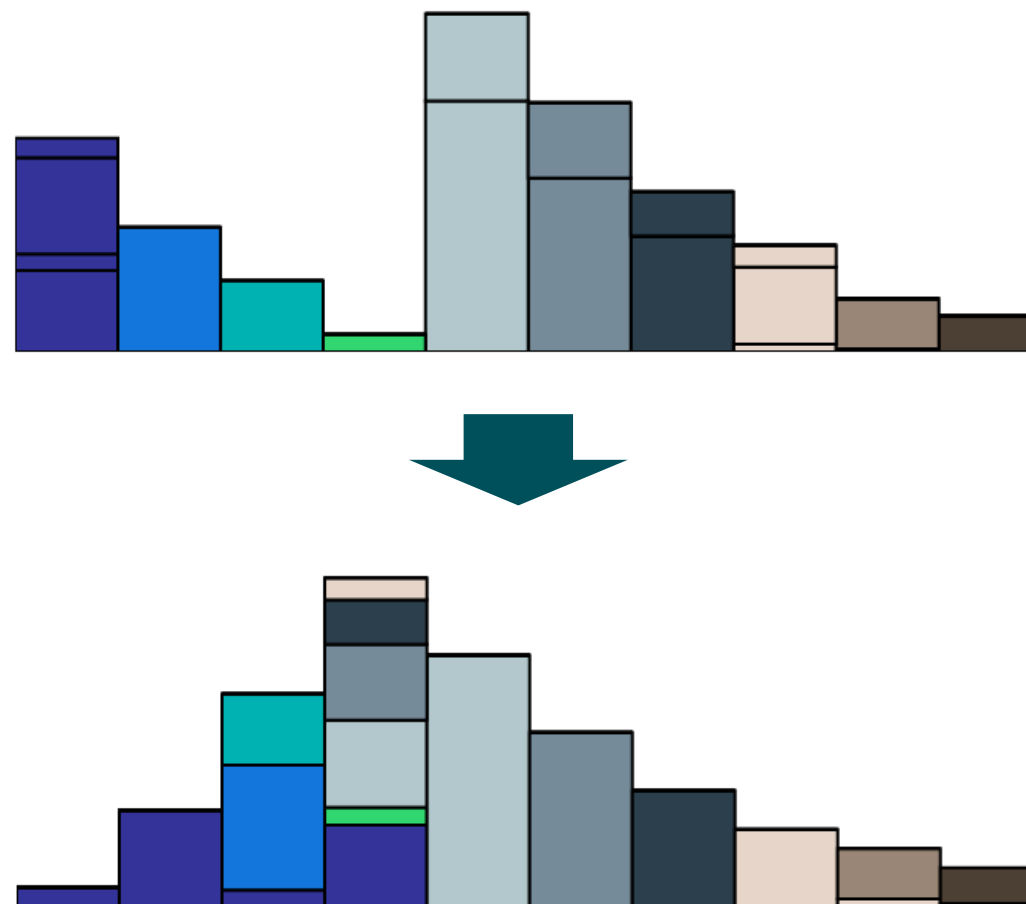
- $\mu$  and  $\nu$ : Probability distributions
- $\Gamma(\mu, \nu)$ : A set of possible joint distributions



# $p$ -Wasserstein Distance

A discrete setup:

The **minimum** amount of the **sum of mass $\times$ distance** to match one distribution to the other.



# $p$ -Wasserstein Distance

When the two distributions are normal distributions:

$$\mu = \mathcal{N}(m_1, C_1) \text{ and } \nu = \mathcal{N}(m_2, C_2),$$

2-Wasserstein distance becomes the following:

$$W(\mu, \nu) = \|m_1 - m_2\|_2^2 + \text{Tr} \left( C_1 + C_2 - 2 \left( C_2^{1/2} C_1 C_2^{1/2} \right)^{1/2} \right).$$

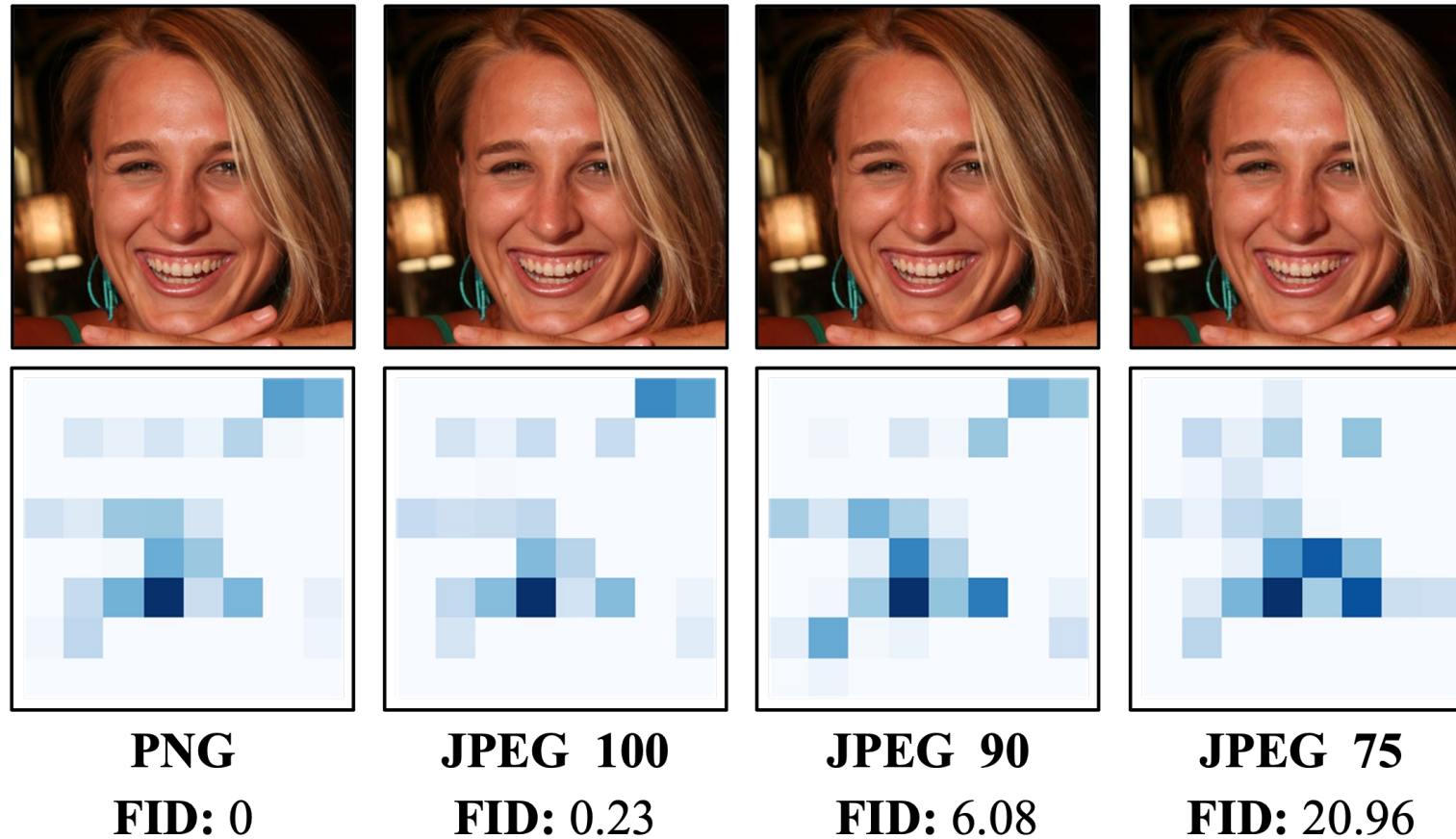
# Fréchet Inception Distance (FID)

Compare the distributions of the real images and generated images in the latent space of **Inception v3<sup>1</sup>** network.

- Map all real and generated images to the latent space of the **Inception v3** network.
- Model each distribution as a **normal** distribution and compute the mean and covariance.
- Compute the 2-Wasserstein (Fréchet) distance.

# Fréchet Inception Distance (FID)

Is FID a perfect metric for generative models?



# Codebase Structure

# Codebase Structure

```
AIC313-CS378-Project-FastGen/
├── model.py           # NFE-specific model classes (students SHOULD modify subclasses)
├── dataset.py         # Pokemon dataset and DataLoader API (provided, DO NOT modify)
├── download_dataset.py # Kaggle dataset download utility (provided, DO NOT modify)
├── evaluate.py        # Sample generation and FID evaluation (provided, DO NOT modify)
├── src/
│   ├── __init__.py
│   └── utils.py      # Parameter counting utilities (provided, DO NOT modify)
└── data/
    ├── pokemon-generation-one-22k/
    │   ├── train_split.txt    # Fixed training split (provided, DO NOT modify)
    │   ├── val_split.txt     # Fixed validation split (provided, DO NOT modify)
    │   └── category_to_id.json # Fixed category-to-ID mapping (provided, DO NOT modify)
```

- **DO NOT modify:** Files that must be kept as-is (for fair comparison)
- **SHOULD modify:** Main files for implementing your solution

# Codebase Structure

```
AIC313-CS378-Project-FastGen/
├── model.py                # NFE-specific model classes (students SHOULD modify subclasses)
├── dataset.py              # Pokemon dataset and DataLoader API (provided, DO NOT modify)
├── download_dataset.py     # Kaggle dataset download utility (provided, DO NOT modify)
├── evaluate.py             # Sample generation and FID evaluation (provided, DO NOT modify)
├── src/
│   ├── __init__.py
│   └── utils.py          # Parameter counting utilities (provided, DO NOT modify)
├── data/
│   └── pokemon-generation-one-22k/
│       ├── train_split.txt # Fixed training split (provided, DO NOT modify)
│       ├── val_split.txt   # Fixed validation split (provided, DO NOT modify)
│       └── category_to_id.json # Fixed category-to-ID mapping (provided, DO NOT modify)
```

Keep them fixed

- **DO NOT modify:** Files that must be kept as-is (for fair comparison)
- **SHOULD modify:** Main files for implementing your solution

# Codebase Structure

```
AIC313-CS378-Project-FastGen/
├── model.py           Main part to implement
├── dataset.py
├── download_dataset.py
├── evaluate.py
├── src/
│   ├── __init__.py
│   └── utils.py       # Parameter counting utilities (provided, DO NOT modify)
├── data/
│   └── pokemon-generation-one-22k/
│       ├── train_split.txt    # Fixed training split (provided, DO NOT modify)
│       ├── val_split.txt      # Fixed validation split (provided, DO NOT modify)
│       └── category_to_id.json # Fixed category-to-ID mapping (provided, DO NOT modify)
```

- **DO NOT modify:** Files that must be kept as-is (for fair comparison)
- **SHOULD modify:** Main files for implementing your solution

# Codebase Structure

```
AIC313-CS378-Project-FastGen/
├── model.py           Main part to implement
├── dataset.py
├── download_dataset.py
├── evaluate.py
├── src/
│   ├── __init__.py
│   └── utils.py      # Parameter counting utilities (provided, DO NOT modify)
├── data/
│   └── pokemon-generation-one-22k/
│       ├── train_split.txt    # Fixed training split (provided, DO NOT modify)
│       ├── val_split.txt     # Fixed validation split (provided, DO NOT modify)
│       └── category_to_id.json # Fixed category-to-ID mapping (provided, DO NOT modify)
```

**You can also freely add your own implementation!**

# What to Do

# What to Do

- We provide the model skeleton code and evaluation code.
- Your goal is to **design and implement your own image generation models**, including:
  - **few-step generation framework:** either using exist framework or design your own generative model.
  - **model architecture design:** design the good model backbone.
  - **other training details:** data augmentation, learning rate, optimizer, etc.

# Implement Your Own Model

- You may modify the provided code (`model.py`) and add new files if necessary.
- You must implement the **following two models** used by the evaluation code.
  - `class ModelOneNFE(Model):` Model for 1-NFE generation
  - `class ModelFewNFE(Model):` Model for few-NFE generation
- DO NOT Modify `class Model(nn.Module)`

# Train the Model

- You may freely use your own training pipeline, including model training and dataset preparation.
- You can design your own model, but there are some restrictions:
  - The total parameter size of each model should be less than 60,000,000 (60M).
  - The peak VRAM while training the model should be less than 20GB.
  - Utilizing any pretrained model is prohibited.

# Sampling Images

- You should also implement your model's sampling code (`Model.sample()`) for each models within pre-defined NFE limit.
- NFE: Number of Function Evaluations (=Sampling Steps)
  - Any use of the trained model backbone, **even partially**, will be counted as 1 NFE.

# Evaluation

# Evaluation

- FID scores at NFE=1, and  $\leq 4$ .
- You may compute FID yourself, but TA-measured scores are official.
- TAs will run your code as-is.
- Mid evaluation results will be published on the leaderboard.
- At the mid evaluation, top-k submissions above TA's result earn bonus credit.

# Evaluation

- Final grading will be determined relative to the best score achieved for each metric:
  - You can get total **20 points** from FastGen project.

$$\text{Score} = \max \left( \frac{\text{Your Score}}{\text{Highest Score}} \times 8, 0 \right)$$

# What to Submit

# Overview of Submission

- There are two submission deadlines:  
**the mid evaluation** (*optional*) and **the final submission**.
- The mid evaluation is optional, but **top-k** submissions above the TA's result will earn bonus credit.
- The purpose of the mid evaluation is to check your team's standing among others.

# 1. Mid-term Evaluation Submission

- The FID results will be published in the leaderboard.
- We will provide the TA's FID scores as a yardstick after the mid-term evaluation.
- **Top-k** submissions above the TA's result will earn bonus credit.

# 1. Mid Evaluation Submission

- **Due:** November 2 (Saturday) 23:59 KST.  
(two weeks before the final submission)
- **What to Submit:**
  - **Self-contained source code**
    - Your submission must include the complete codebase.
    - TAs will run your code in their environment with no changes.
  - **Model checkpoint(s) and configs**
    - Checkpoint path: `./checkpoints/{one_nfe.ckpt, few_nfe.ckpt}`.

TAs load from these paths.  
**Different paths will FAIL.**

# Example of Mid Submission

```
team_{team-id:0>2}/
├─ model.py           # model_one_nfe, model_few_nfe, and Model
├─ train.py           # Student training script
├─ dataset.py         # Provided dataset/DataLoader interface
├─ download_dataset.py # Provided dataset downloader
├─ evaluate.py        # Provided evaluation entry point
├─ src/
│   ├── __init__.py
│   └─ utils.py        # Provided parameter-counting utilities
├─ requirements.txt
├─ checkpoints/
│   ├── one_nfe.ckpt   # REQUIRED
│   └─ few_nfe.ckpt   # REQUIRED
└─ <all additional files> # Include every file you implemented
```

## 2. Final Submission

- **Due:** November 15 (Saturday) 23:59 KST.
- **What to Submit:**
  - Same as the mid evaluation submission, **plus a 2-page write-up.**
- **Write-up Guidelines:**
  - No template. Maximum of two A4 pages, excluding references.
  - Contents should include:
    - Technical details: One-paragraph description
    - Training details: Training logs and total training time
    - 8 sample images from the early training phases
    - Citations: All external code, and papers used

# Self-Evaluation Checklist

Before submitting:

- Use TA's `requirements.txt` for compatibility check.
- Code runs end-to-end: Train → Sample → Evaluate without errors.
- Correct checkpoint path set
- Citation ready: All external code/papers in final write-up.

# Grading

Please note that your score may be reduced according to the following rules:

- **Late submission:** Zero score.
- **Missing any required item in the final submission:** Zero score.
- **Missing items in the write-up:** 10% penalty for each.

# Q&A

- Further details are provided in the README.md.
- Any questions?